

Inference Engineering

Assignment — Lecture 1 + 2

A simple, plain-language walkthrough of how LLM inference works, how much memory models need, and how to pick the right hardware & software.

What's inside

- ✓ Theory: why inference is slow, memory math, the inference stack
- ✓ Hardware: comparing two real GPUs
- ✓ Software: two serving frameworks explained
- ✓ Coding: reading `nvidia-smi` output

1 Theory

1.1 1. Why is inference memory bandwidth-bound important?

In one sentence: Generating text is slow mainly because the GPU spends most of its time *waiting to fetch the model's weights from memory*, not because it runs out of raw calculating power.

Think of the GPU like a chef, and the model's weights like ingredients stored in a fridge. To cook one dish (generate one token), the chef needs to grab *every single ingredient* out of the fridge, even if they only use each one very briefly. If the fridge is far away or slow to open, the chef spends most of their time walking back and forth instead of actually cooking.

That's exactly what happens during text generation:

- To produce just *one* new word, the GPU has to read the *entire* model (all its weights) from memory.
- Reading that much data takes much longer than the tiny amount of math needed once the data has arrived.
- So the "cooking" (math) is fast, but the "fridge trips" (memory reads) are the real bottleneck.

The technical version — Arithmetic Intensity:

$$\text{Arithmetic Intensity} = \frac{\text{Amount of math done}}{\text{Amount of data read from memory}}$$

Every GPU has a top speed for math (FLOPs/s) and a top speed for reading memory (bytes/s). The point where these two speeds match is called the **ridge point**. If a task's arithmetic intensity is *below* the ridge point, it is **memory-bound** (limited by data-fetching speed). Generating text, one word at a time, almost always falls below this line.

Why this matters in practice:

- Buying a GPU with more raw compute power (TFLOPs) barely helps if it's already stuck waiting on memory.
- What actually helps: (1) **shrinking the model** so there's less to read (quantization), or (2) **serving many users at once** so each memory trip does more useful work (batching).
- This is why **memory bandwidth** (GB/s) is one of the most important specs to check when buying or renting a GPU for inference — often more important than raw TFLOPs.

1.2 2. How do we calculate the memory needed to store a model, and how are models stored?

In one sentence: A model is just a huge list of numbers (called *parameters*), and the memory it needs is simply *how many numbers there are* $\times$ *how many bytes each number takes up*.

How models are stored: An LLM is built from layers (attention layers, feed-forward layers, etc.), and each layer is made of matrices full of numbers called **parameters** or **weights**. These numbers are saved to disk in formats like `safetensors`, and each number is stored using a fixed number of bytes, depending on the "precision" chosen:

Precision	Bytes per number	Where it's used
FP32 (full)	4 bytes	Mostly for training
FP16 / BF16 (half)	2 bytes	Most common for serving
INT8 / FP8	1 byte	Common "quantized" option
INT4 (4-bit)	0.5 bytes	Aggressive compression for small GPUs

Simple formula:

Memory needed = Number of parameters $\times$ Bytes per parameter

Example

A 7-billion-parameter model saved in FP16 (2 bytes each):

$$7,000,000,000 \times 2 \text{ bytes} = 14,000,000,000 \text{ bytes} \approx 14 \text{ GB}$$

But that's not the whole story. When the model actually *runs*, it also needs extra memory for:

- **KV cache** — a "memory pad" that stores information about every word processed so far, so the model doesn't have to redo work. This grows the longer the conversation gets, and the more users are served at once.
- **Activations** — short-lived scratch space used while the model is computing.
- **Overhead** — the usual bit of extra memory the software itself needs to run.

A simple rule of thumb: **take the weight memory and add about 20% extra** for overhead, then budget separately for the KV cache based on how many users and how much context you expect.

1.3 3. The layers of inference, explained simply

In one sentence: Serving an AI model to real users isn't just "running the model" — it's a stack of layers, kind of like a restaurant needs a kitchen, a menu, waiters, and a manager, not just a chef.

- 1. The Hardware Layer** — the GPUs themselves (their memory size, memory speed, how they connect to each other).
Why a company cares: This is the biggest cost in the whole system. Pick the wrong GPU, and you either can't run your model at all, or you're paying way more than you need to.
- 2. The Model/Weights Layer** — how the model is stored and compressed (full precision vs. quantized).
Why a company cares: Shrinking the model (quantization) can let you run 2–4x cheaper hardware for almost the same quality — directly improving profit margins.
- 3. The Inference Engine Layer** — the actual software that runs the model efficiently (e.g., vLLM, TensorRT-LLM, TGI). It handles clever tricks like batching many requests together and managing memory smartly.
Why a company cares: A good engine can serve 5–10x more users on the exact same hardware compared to a naive setup — this is where most of the "free performance" comes from.
- 4. The Serving/Orchestration Layer** — the layer that takes incoming requests, spreads them across many GPUs, and scales up or down automatically when traffic changes.
Why a company cares: This is what keeps the product online and responsive during busy periods, which is essential for keeping customers happy.
- 5. The Application Layer** — the APIs, safety filters, and tools that developers actually use to build features on top of the model.
Why a company cares: This decides how fast internal teams can ship new features, and it's where safety and compliance rules get enforced.
- 6. The Monitoring & Cost Layer** — dashboards and logs that track speed, errors, and how much everything costs.
Why a company cares: Without this, a company has no idea if they're wasting money or about to have an outage — GPUs are expensive, so blind spots here get costly fast.

★ Key Idea

If even one of these layers is weak — say, a slow inference engine, or the wrong GPU — it can bottleneck the *entire* system, no matter how good the underlying model is. That's why "inference engineering" is its own discipline.

2 Hardware & Software Exploration

2.1 4. Comparing two real GPUs

We picked two very different GPUs: the **NVIDIA A100 80GB** (a big, expensive data-center GPU) and the **NVIDIA RTX 4090** (a cheaper, consumer-grade GPU popular for smaller projects).

Spec	A100 80GB (SXM)	RTX 4090
VRAM (memory size)	80 GB	24 GB
Memory Bandwidth (speed)	≈2.0 TB/s	≈1.0 TB/s
Cloud Rental Cost	≈ \$1.05–1.49/hr	≈ \$0.35–0.45/hr

Simple way to estimate the biggest model a GPU can hold:

$$\text{Max parameters (billions)} \approx \frac{\text{Usable VRAM (GB)}}{\text{Bytes per parameter}}$$

(We use about 85–90% of total VRAM as "usable," since some memory is needed for overhead.)

Biggest model that fits	A100 80GB	RTX 4090
At 16-bit precision	≈34–36B params	≈10–11B params
At 4-bit precision	≈136–144B params	≈40–42B params

Plain-language takeaway: The A100 can comfortably run a huge 70B model once shrunk to 4-bit, while the much cheaper RTX 4090 is a great fit for smaller 7B–8B models at full quality, or a shrunk-down 30B model at 4-bit. These numbers only cover the model weights — real-world usage needs a bit more room for the "memory pad" (KV cache) mentioned earlier.

2.2 5. Two serving frameworks we didn't cover in class

(a) Hugging Face TGI (Text Generation Inference)

What it is, simply: A ready-made server built by Hugging Face that lets you deploy almost any model from their website with one command. It handles the technical stuff automatically — batching multiple requests, streaming text back word-by-word, and squeezing out good performance.

Best used for: Teams who are already using Hugging Face's tools and want something simple, well-documented, and battle-tested. It also works on more types of hardware than just NVIDIA (like AMD chips).

When to choose it over vLLM/SGLang/TensorRT-LLM: When ease of setup and Hugging Face Hub integration matter more than getting the absolute fastest speed.

Good to know: Hugging Face has now put TGI into "maintenance mode" and recommends vLLM or SGLang for new projects, so it's more of a solid legacy choice than a first pick today.

(b) LMDeploy

What it is, simply: A serving toolkit built by the team behind the InternLM models, known for being *very* fast, especially when the model has been shrunk down (quantized). Its special engine, called **TurboMind**, is written in C++ instead of Python, which cuts out a lot of "wasted" overhead.

Best used for: Getting the maximum possible speed out of NVIDIA GPUs, particularly for 4-bit quantized models (up to 2.4x faster than full precision).

When to choose it over vLLM/SGLang/TensorRT-LLM: When your model is well-supported by TurboMind and raw throughput on NVIDIA hardware is the top priority. It's a weaker choice if you need support for many different model types or non-NVIDIA chips, since its optimizations are narrower and NVIDIA-specific.

3 Coding Exploration

3.1 6. Reading `nvidia-smi` output

In one sentence: `nvidia-smi` is like a dashboard for your GPU — it shows what's installed, how full its memory is, how hot it's running, and what programs are using it right now.

Below is an example of what the output looks like when you run it:

```
NVIDIA-SMI 550.90   Driver:  550.90.07   CUDA Version:  12.4
GPU:  0    A100-SXM4-80GB   Bus-Id:  00000000:00:04.0
Fan:   30%   Temp:  45C    Power:  68W / 400W
Memory-Usage:  21504MiB / 81920MiB   GPU-Util:  37%
Processes:
  PID 12345   python (vllm serve ...)   Memory:  20800MiB
```

Here are 7 things this tells us, explained simply:

- 1. Driver & CUDA Version** — the software “translator” installed between your GPU and programs like PyTorch. *Why it matters:* if this version is too old, many AI tools simply refuse to run.
- 2. GPU Name** — exactly which GPU model you’re using (e.g. A100-SXM4-80GB). *Why it matters:* confirms you’re actually running on the hardware you paid for, and tells you your memory/speed limits.
- 3. Memory-Usage (21504MiB / 81920MiB)** — how much of the GPU’s memory is currently being used out of the total. *Why it matters:* this is the #1 number to watch — if it gets too close to the total, you’ll get an “out of memory” crash.
- 4. GPU-Util (37%)** — what percentage of the time the GPU was actually crunching numbers (instead of waiting). *Why it matters:* a low number during heavy use is a sign the GPU is stuck waiting on memory, not on math (ties back to Question 1!).
- 5. Power Usage (68W / 400W)** — how much electricity the GPU is drawing right now vs. its maximum. *Why it matters:* power draw close to the GPU is actually working hard; power draw far below the max often means it’s sitting idle, waiting for data.
- 6. Temperature & Fan Speed (45C, 30%)** — how hot the card is running. *Why it matters:* if a GPU gets too hot, it automatically slows itself down (throttling) to protect itself — which quietly hurts performance without any error message.
- 7. Processes list** — shows exactly which programs are using the GPU and how much memory each one is holding. *Why it matters:* super useful for finding a leftover (“zom-

bie”) process that’s hogging memory even though nothing looks like it’s running.

★ Key Idea

This is a good warm-up for next class, which goes deeper into *how* GPU commands actually get scheduled and run under the hood.